



RAJALAKSHMI INSTITUTE OF TECHNOLOGY
(An Autonomous Institution, Affiliated to Anna University, Chennai)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ACADEMIC YEAR 2025 - 2026

SEMESTER IV

AL23421 - MACHINE LEARNING LABORATORY

MINI PROJECT REPORT

REGISTER NUMBER	2117240030002
NAME	AJAY M
PROJECT TITLE	Product Recommendation Using K-Nearest Neighbors Algorithm
DATE OF SUBMISSION	
FACULTY IN-CHARGE	Ms. M. Divya

Signature of Faculty In-charge

Signature of HoD

INTRODUCTION

Machine Learning is a rapidly evolving field of Artificial Intelligence that focuses on building systems capable of learning from data and making intelligent decisions without explicit programming. It has become an essential tool for solving complex real-world problems across various domains such as e-commerce, healthcare, finance, and personalized marketing. Among the various machine learning algorithms, the K-Nearest Neighbors (KNN) algorithm is widely used due to its simplicity, effectiveness, and ability to provide accurate recommendations based on similarity measures.

K-Nearest Neighbors (KNN) is a supervised learning algorithm that works on the principle of similarity. It identifies the 'K' closest data points (neighbors) to a given input based on a distance metric such as Euclidean distance and makes predictions based on these neighbors. Unlike other algorithms, KNN does not build an explicit model but instead stores the dataset and makes decisions at runtime, making it highly suitable for recommendation systems where similarity between users or products plays a crucial role.

In this project, the KNN algorithm is applied to develop a Product Recommendation System. Recommending relevant products to users is one of the key challenges in modern e-commerce platforms. With the vast availability of products, users often find it difficult to discover items that match their preferences. A recommendation system helps overcome this challenge by suggesting products based on user behavior, preferences, and similarity with other users or items.

The system aims to recommend products to users based on features such as user ratings, purchase history, product categories, and user preferences. By analyzing these factors, the model identifies similar users or products and generates personalized recommendations. This approach ensures that users receive relevant suggestions, thereby enhancing their overall experience.

The problem addressed in this project is the lack of efficient and personalized recommendation systems that can handle large volumes of data and provide accurate suggestions. Traditional recommendation approaches may fail to capture user preferences effectively, leading to irrelevant suggestions. This project proposes a data-driven solution that leverages the KNN algorithm to provide meaningful and personalized product recommendations.

The main objective of this project is to design and implement a KNN-based recommendation model using a dataset that reflects real-world user-product interactions. The model is trained using relevant features and evaluated using performance metrics such as Accuracy, Precision, Recall, and F1-Score to ensure the effectiveness of the recommendations.

In conclusion, this project demonstrates the practical application of machine learning in recommendation systems. It highlights how algorithms like KNN can be used to analyze user behavior and deliver personalized product suggestions, ultimately improving user satisfaction and supporting better decision-making in e-commerce platforms.

GOAL

The primary goal of this project is to develop an efficient and reliable product recommendation system using the K-Nearest Neighbors (KNN) algorithm. The system is designed to take user input in the form of a product name, analyze the similarity between products using distance-based metrics, and generate relevant product recommendations. By leveraging historical product data, the system aims to provide accurate and meaningful suggestions that improve the overall user experience.

The expected outcome of this project is a well-implemented KNN-based recommendation model that can efficiently identify similar products and deliver quick results through a web-based interface. The system should be capable of handling real-world datasets and producing consistent recommendations without requiring complex computations or manual filtering. It should also demonstrate how simple machine learning algorithms can be effectively used in practical applications.

Another important goal of this project is to understand and analyze product similarity patterns based on available data. By examining how different products are related to each other, the system helps in identifying key factors that influence recommendations. This insight is valuable for improving recommendation quality and understanding user preferences in an e-commerce environment.

In addition to generating recommendations, the project aims to integrate the machine learning model with a web application using Flask and HTML. This ensures that the system is not only functional but also user-friendly and accessible. The interactive interface allows users to easily input product names and receive recommendations in real time.

In conclusion, the goal of this project is not only to build an accurate recommendation model but also to develop a practical, scalable, and intelligent system that can be applied to real-world applications, enhancing product discovery and supporting better user decision-making.

THEORETICAL BACKGROUND

Background of the Problem

Efficient product discovery in modern digital platforms requires intelligent systems that can analyze large volumes of data and provide relevant recommendations. In e-commerce environments, users are often presented with thousands of products, making it difficult to identify items that match their preferences. Product recommendation systems address this challenge by analyzing user behavior, product features, and similarity patterns to suggest items that are most relevant. Without such systems, users may experience information overload, leading to poor decision-making and reduced satisfaction.

Traditional recommendation approaches rely on manual filtering, keyword-based search, or simple rule-based systems, which often fail to capture the deeper relationships between products. These methods are limited in their ability to adapt to changing user preferences and

cannot effectively handle large and complex datasets. As a result, there is a need for intelligent systems that can automatically identify patterns and similarities among products to generate accurate recommendations.

Machine learning provides an effective solution to this problem by enabling systems to learn from data and make predictions based on similarity measures. The K-Nearest Neighbors (KNN) algorithm is a widely used technique for recommendation tasks, as it identifies products that are most similar to a given input based on distance metrics such as Euclidean distance or cosine similarity. By leveraging KNN, the system can recommend products that closely match user interests, thereby improving the efficiency and accuracy of the recommendation process.

By automating the recommendation process, the system reduces manual effort, enhances user experience, and supports better decision-making. This approach demonstrates how machine learning techniques can be applied to real-world problems, providing scalable and efficient solutions for modern e-commerce applications.

K-Nearest Neighbors (KNN) Algorithm

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm used for both classification and recommendation tasks. It is a simple, instance-based learning method that works by finding the closest data points (neighbors) to a given input based on a similarity or distance measure. Unlike other algorithms, KNN does not require a separate training phase; instead, it stores the entire dataset and performs computations during prediction.

In KNN, when a new input is given, the algorithm calculates the distance between the input and all data points in the dataset. It then selects the top **K** nearest neighbors and makes a prediction based on these neighbors. In recommendation systems, KNN identifies similar items or users and suggests products that are most closely related to the input.

The similarity in KNN is commonly calculated using distance metrics such as Euclidean distance:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

where:

- **d(x, y)** = Distance between two data points
- **x_i, y_i** = Feature values of the data points
- **n** = Number of features

For recommendation, the output is based on selecting the most similar items among the K nearest neighbors. The choice of **K** plays an important role in determining the performance of the model. A smaller value of K may lead to noise sensitivity, while a larger value may reduce the influence of closer neighbors.

KNN is simple, effective, and widely used in recommendation systems due to its ability to capture similarity patterns directly from data without complex model training

ALGORITHM EXPLANATION WITH EXAMPLE

Algorithm Explanation

The K-Nearest Neighbors (KNN) algorithm is a supervised machine learning technique used for classification and recommendation tasks. In this project, it is used to recommend similar products based on features such as price, ratings, and other product attributes. The algorithm works by comparing a selected product with all other products in the dataset and identifying those that are most similar.

KNN operates on the principle of similarity, where similar products are located close to each other in the feature space. When a user inputs a product, the algorithm calculates the distance between the input product and all other products using a distance metric such as Euclidean distance. Based on this, it selects the top K nearest products and recommends them. Unlike other algorithms, KNN does not require a training phase and directly uses the dataset during prediction.

The distance function used in KNN is given by:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

where:

- $d(x, y)$ = Distance between two products
- x_i, y_i = Feature values of the products
- n = Number of features

Steps of the Algorithm

1. Collect the product dataset containing features like price, rating, etc.
 2. Preprocess the data by handling missing values and normalizing features
 3. Convert product information into numerical feature vectors
 4. Choose the value of K (number of nearest neighbors)
 5. Calculate the distance between the input product and all other products
 6. Sort the distances in ascending order
 7. Select the top K nearest products
 8. Recommend these products to the user
 9. Display the results through the web interface
-

PRODUCT RECOMMENDATION USING KNN ALGORITHM

Sample Data

Product	Price (₹)	Rating	Category
Laptop A	50000	4.5	Electronics
Laptop B	52000	4.6	Electronics
Mobile C	15000	4.2	Electronics
Tablet D	30000	4.3	Electronics

Explanation

Consider a user selecting a product such as Laptop A with a price of ₹50,000 and a rating of 4.5 in the Electronics category. The K-Nearest Neighbors (KNN) algorithm processes this input by comparing it with all other products available in the dataset. It calculates the similarity between the selected product and other products using features like price and rating. Products that have similar values for these features will have smaller distances and are considered closer neighbors, while those with very different values will have larger distances.

After computing the distances, the algorithm sorts all products based on their similarity to the selected product. It then selects the top K nearest neighbors (for example, $K = 3$), which are the most similar products. In this case, products like Laptop B and Laptop E may be identified as very close matches due to their similar price range and ratings, while a product like Tablet D may also be included as a moderately similar option. These selected products are then recommended to the user. This process demonstrates how KNN effectively identifies similar products and provides relevant recommendations, helping users make better and quicker decisions.

This shows that the KNN algorithm identifies and recommends products based on feature similarity, helping users discover items that closely match their preferences.

PRODUCT RECOMMENDATION USING KNN ALGORITHM

IMPLEMENTATION CODE AND DATASET USED

➤ Dataset Name	➤ Product Recommendation Dataset
➤ Domain	➤ E-commerce / Recommendation Systems
➤ Objective	➤ Recommendation (Suggest similar products using KNN)
➤ Data Source	➤ Dataset obtained from Kaggle or custom dataset
➤ Number of Samples	➤ ~1000–5000 products (can be scaled)
➤ Data Type	➤ Tabular / Time-Series
➤ Instance Description	➤ Each record represents a product with attributes like price, rating, and category
➤ Features (Inputs)	➤ Product Name, Price, Rating, Category, Product ID
➤ Feature Types	➤ Numerical (Price, Rating), Categorical (Product Name, Category)
➤ Target Variable	➤ No explicit target (similarity-based recommendation)
➤ Number of Outputs	➤ Top K recommended products
➤ Libraries Used	➤ pandas, numpy, scikit-learn, Flask, HTML/CSS
➤ Model Used	➤ K-Nearest Neighbors (KNN) using NearestNeighbors
➤ Evaluation Metrics	➤ Recommendation accuracy (optional), similarity score, user relevance
➤ License	➤ Depends on dataset (Kaggle datasets are generally open for academic use)
➤ Version	➤ Initial version (basic recommendation system)

Dataset link: <https://www.kaggle.com/datasets/kartikeybartwal/ecomerce-product-recommendation-dataset>

Dataset Description

The dataset used in this project contains information related to products available in an e-commerce environment. It includes various product attributes such as product name, price, rating, category, and other relevant features that help in identifying similarities between products. Based on these input features, the system recommends similar products using the K-Nearest Neighbors (KNN) algorithm.

The dataset is obtained from platforms like Kaggle and is structured in a tabular format, making it suitable for machine learning applications. Each row in the dataset represents a specific product with its associated features. The input features capture important characteristics that influence product similarity, while there is no explicit target variable since the system is based on similarity rather than prediction. This dataset enables the model to analyze relationships between products and generate relevant recommendations for users.

Python Code:

app.py

```

"""
KNN Product Recommendation System
Flask application with scikit-learn KNN implementation.
"""

from flask import Flask, render_template, request, jsonify
import numpy as np
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import MinMaxScaler
import json
app = Flask(__name__)
# SAMPLE PRODUCT DATASET
# Features: price (USD), rating (1-5), category_id,
#           popularity (0-100), discount_pct (0-100)
# _____
PRODUCTS = [
    {"id": 1, "name": "Sony WH-1000XM5 Headphones", "category": "Electronics", "price":
349.99, "rating": 4.8, "popularity": 95, "discount": 10, "image_icon": "🎧", "description":
"Industry-leading noise cancellation with 30-hr battery."},
    {"id": 2, "name": "Kindle Paperwhite", "category": "Electronics", "price": 139.99,
"rating": 4.7, "popularity": 88, "discount": 15, "image_icon": "📖", "description": "Adjustable
warm light, waterproof, weeks of battery."},
    {"id": 3, "name": "Nike Air Max 270", "category": "Footwear", "price": 150.00,
"rating": 4.5, "popularity": 90, "discount": 5, "image_icon": "👟", "description": "Max Air
cushioning with a modern silhouette."},

```

PRODUCT RECOMMENDATION USING KNN ALGORITHM

{ "id": 4, "name": "Instant Pot Duo 7-in-1", "category": "Kitchen", "price": 89.99, "rating": 4.7, "popularity": 92, "discount": 20, "image_icon": "🍲", "description": "Pressure cooker, slow cooker, rice cooker and more." },

{ "id": 5, "name": "Levi's 501 Original Jeans", "category": "Clothing", "price": 69.50, "rating": 4.4, "popularity": 85, "discount": 0, "image_icon": "👖", "description": "Iconic straight-fit jeans, timeless since 1873." },

{ "id": 6, "name": "Apple AirPods Pro (2nd Gen)", "category": "Electronics", "price": 249.00, "rating": 4.8, "popularity": 97, "discount": 8, "image_icon": "🎧", "description": "Adaptive Transparency, Personalized Spatial Audio." },

{ "id": 7, "name": "Yoga Mat Premium Cork", "category": "Sports", "price": 45.00, "rating": 4.6, "popularity": 78, "discount": 10, "image_icon": "🧘", "description": "Natural cork surface, anti-slip, eco-friendly." },

{ "id": 8, "name": "Dyson V15 Detect", "category": "Home", "price": 699.99, "rating": 4.7, "popularity": 82, "discount": 5, "image_icon": "🧹", "description": "Laser dust detection, HEPA filtration, 60-min runtime." },

{ "id": 9, "name": "The Alchemist – Hardcover", "category": "Books", "price": 14.99, "rating": 4.6, "popularity": 94, "discount": 0, "image_icon": "📖", "description": "Paulo Coelho's timeless tale of following your dream." },

{ "id": 10, "name": "Whey Protein Isolate 5lb", "category": "Sports", "price": 59.99, "rating": 4.5, "popularity": 87, "discount": 12, "image_icon": "💪", "description": "25g protein per serving, low carb, fast absorption." },

{ "id": 11, "name": "Nespresso Vertuo Pop", "category": "Kitchen", "price": 119.00, "rating": 4.6, "popularity": 80, "discount": 18, "image_icon": "☕", "description": "Centrifusion tech, 5 cup sizes, milk frother included." },

{ "id": 12, "name": "Patagonia Nano Puff Jacket", "category": "Clothing", "price": 229.00, "rating": 4.7, "popularity": 76, "discount": 0, "image_icon": "🧥", "description": "Lightweight, windproof, 60g PrimaLoft insulation." },

{ "id": 13, "name": "Logitech MX Master 3S", "category": "Electronics", "price": 99.99, "rating": 4.8, "popularity": 89, "discount": 7, "image_icon": "🖱️", "description": "8K DPI sensor, MagSpeed scroll, silent clicks." },

{ "id": 14, "name": "Adidas Ultraboost 23", "category": "Footwear", "price": 189.99, "rating": 4.6, "popularity": 88, "discount": 10, "image_icon": "👟", "description": "BOOST midsole, PRIMEKNIT+ upper, Continental rubber." },

{ "id": 15, "name": "Vitamix 5200 Blender", "category": "Kitchen", "price": 449.95, "rating": 4.8, "popularity": 75, "discount": 0, "image_icon": "🍹", "description": "Aircraft-grade stainless blades, 7-year warranty." },

{ "id": 16, "name": "Moleskine Classic Notebook", "category": "Books", "price": 19.95, "rating": 4.5, "popularity": 86, "discount": 5, "image_icon": "📓", "description": "Hard cover, ruled pages, elastic closure, ribbon." },

{ "id": 17, "name": "TRX Suspension Trainer", "category": "Sports", "price": 149.95, "rating": 4.6, "popularity": 72, "discount": 8, "image_icon": "🏋️", "description": "Full-body workout, door anchor included, military-grade." },

PRODUCT RECOMMENDATION USING KNN ALGORITHM

```
{ "id": 18, "name": "Roomba j7+ Robot Vacuum", "category": "Home", "price":
599.99, "rating": 4.5, "popularity": 79, "discount": 15, "image_icon": "🤖", "description":
"Obstacle avoidance, self-emptying base, smart mapping."},
]
CATEGORY_MAP = {
    "Electronics": 1, "Footwear": 2, "Kitchen": 3,
    "Clothing": 4, "Sports": 5, "Home": 6, "Books": 7,
}
def build_feature_matrix(products):
    """
    Convert product list into a numeric feature matrix.
    Features: [price, rating, category_id, popularity, discount_pct]
    """
    matrix = []
    for p in products:
        matrix.append([
            p["price"],
            p["rating"],
            CATEGORY_MAP.get(p["category"], 0),
            p["popularity"],
            p["discount"],
        ])
    return np.array(matrix, dtype=float)
# Pre-build and cache the KNN model at startup
_scaler = MinMaxScaler()
_feature_matrix = build_feature_matrix(PRODUCTS)
_scaled_matrix = _scaler.fit_transform(_feature_matrix)
# k=6 so we can return top-5 excluding the query point itself when it matches
_knn_model = NearestNeighbors(n_neighbors=6, metric="euclidean", algorithm="auto")
_knn_model.fit(_scaled_matrix)
def get_recommendations(user_prefs: dict, n: int = 5):
    """
    -----

    user_prefs : dict Keys: max_price, min_rating, category, min_popularity, min_discount
    n           : int  Number of results to return (default 5)
    """
    # --- Build query vector ---
    category_id = CATEGORY_MAP.get(user_prefs.get("category", ""), 0)
    # If no category selected (0), use midpoint so it doesn't skew results
    if category_id == 0:
        category_id = 4.0

    query = np.array([
        float(user_prefs.get("max_price", 300)),
```

PRODUCT RECOMMENDATION USING KNN ALGORITHM

```
float(user_prefs.get("min_rating", 4.0)),
float(category_id),
float(user_prefs.get("min_popularity", 70)),
float(user_prefs.get("min_discount", 0)),
]])

# --- Scale using the pre-fit scaler ---
query_scaled = _scaler.transform(query)
# --- KNN query ---
k = min(n + 1, len(PRODUCTS))
distances, indices = _knn_model.kneighbors(query_scaled, n_neighbors=k)

results = []
for dist, idx in zip(distances[0], indices[0]):
    product = PRODUCTS[idx].copy()
    # Convert Euclidean distance → similarity percentage (0–100)
    # Smaller distance = more similar
    similarity = round(max(0, (1 - dist) * 100), 1)
    product["similarity"] = similarity
    product["distance"] = round(float(dist), 4)
    # Apply hard filters AFTER KNN (KNN finds geometrically similar;
    # filters remove anything that strictly violates user constraints)
    max_price = float(user_prefs.get("max_price", 99999))
    min_rating = float(user_prefs.get("min_rating", 0))
    pref_category = user_prefs.get("category", "")
    if product["price"] > max_price:
        continue
    if product["rating"] < min_rating:
        continue
    if pref_category and product["category"] != pref_category:
        continue # only filter by category when explicitly chosen
    results.append(product)
    if len(results) >= n:
        break
# If strict filters left us with too few, relax category filter and retry
if len(results) < 2:
    results = []
    for dist, idx in zip(distances[0], indices[0]):
        product = PRODUCTS[idx].copy()
        similarity = round(max(0, (1 - dist) * 100), 1)
        product["similarity"] = similarity
        product["distance"] = round(float(dist), 4)
        results.append(product)
        if len(results) >= n:
            break
return results
```

PRODUCT RECOMMENDATION USING KNN ALGORITHM

```
@app.route("/")
def index():
    categories = list(CATEGORY_MAP.keys())
    return render_template("index.html", categories=categories)
@app.route("/recommend", methods=["POST"])
def recommend():
    try:
        prefs = {
            "max_price": request.form.get("max_price", 500),
            "min_rating": request.form.get("min_rating", 4.0),
            "category": request.form.get("category", ""),
            "min_popularity": request.form.get("min_popularity", 70),
            "min_discount": request.form.get("min_discount", 0),
            "n_results": int(request.form.get("n_results", 5)),
        }
        n = max(1, min(int(prefs["n_results"]), 10))
        recommendations = get_recommendations(prefs, n=n)
        categories = list(CATEGORY_MAP.keys())
        return render_template(
            "index.html",
            categories=categories,
            recommendations=recommendations,
            prefs=prefs,
            total_products=len(PRODUCTS),
        )
    except Exception as e:
        categories = list(CATEGORY_MAP.keys())
        return render_template(
            "index.html",
            categories=categories,
            error=f"Something went wrong: {str(e)}",
        ), 500
@app.route("/api/recommend", methods=["POST"])
def api_recommend():
    """JSON API endpoint for programmatic access."""
    data = request.get_json(force=True) or {}
    recommendations = get_recommendations(data, n=int(data.get("n_results", 5)))
    return jsonify({"status": "ok", "count": len(recommendations), "results": recommendations})
if __name__ == "__main__":
    print("🚀 KNN Recommender running at http://127.0.0.1:5000")
    app.run(debug=True, port=5000)
```

OUTPUT

ProximaRec

KNN - scikit-learn

MACHINE LEARNING · RECOMMENDATION ENGINE

Find products that *match* your preferences.

K-Nearest Neighbors computes Euclidean distance across price, rating, category, popularity, and discount to surface the most geometrically similar products to your preferences.

01 Your Preferences

Max BudgetUSD

\$300

951000

Minimum Ratingout of 5

★ 4.0+

Categoryoptional

— All categories —

Min Popularity Score0 - 100

70

0100

Minimum Discount%

0

Results to Return1 - 10

3

5

8

10

Run KNN Search

02 Recommendations

KNN

Set your preferences and run a search to discover products closest to you in feature space.

PRODUCT RECOMMENDATION USING KNN ALGORITHM

ProximaRec

KNN - scikit-learn

80 products indexed

MACHINE LEARNING · RECOMMENDATION ENGINE

Find products that *match* your preferences.

K-Nearest Neighbors computes Euclidean distance across price, rating, category, popularity, and discount to surface the most geometrically similar products to your preferences.

01 Your Preferences

Max Budget

USD

\$ 500

\$5

\$2000

Minimum Rating

out of 5

★ 4.5+

Category

optional

Sports

Min Popularity Score

0 - 100

70

0

100

Minimum Discount

%

10

Results to Return

1 - 10

3

5

8

10

Run KNN Search

02 3 Recommendations

Sorted by KNN similarity ↓

TRX Suspension Trainer

Full-body workout, door anchor included, military-grade.

PRICE \$149.95

RATING ★ 4.6

POPULARITY 72/100

KNN SIMILARITY 41.7%

Yoga Mat Premium Cork

Natural cork surface, anti-slip, eco-friendly.

PRICE \$45.00

RATING ★ 4.6

POPULARITY 78/100

KNN SIMILARITY 22.1%

Whey Protein Isolate 5lb

25g protein per serving, low carb, fast absorption.

PRICE \$59.99

RATING ★ 4.6

POPULARITY 67/100

KNN SIMILARITY 5.9%

ProximaRec

KNN - scikit-learn

80 products indexed

MACHINE LEARNING · RECOMMENDATION ENGINE

Find products that *match* your preferences.

K-Nearest Neighbors computes Euclidean distance across price, rating, category, popularity, and discount to surface the most geometrically similar products to your preferences.

01 Your Preferences

Max Budget

USD

\$ 500

\$5

\$2000

Minimum Rating

out of 5

★ 4.7+

Category

optional

Electronics

Min Popularity Score

0 - 100

100

0

100

Minimum Discount

%

10

Results to Return

1 - 10

3

5

8

10

Run KNN Search

02 4 Recommendations

Sorted by KNN similarity ↓

Sony WH-1000XM5 Headphones

Industry-leading noise cancellation with 30-hr battery.

PRICE \$349.99

RATING ★ 4.8

POPULARITY 95/100

KNN SIMILARITY 61.2%

Apple AirPods Pro (2nd Gen)

Adaptive Transparency, Personalized Spatial Audio.

PRICE \$249.00

RATING ★ 4.8

POPULARITY 97/100

KNN SIMILARITY 53.0%

Kindle Paperwhite

Adjustable warm light, waterproof, weeks of battery.

PRICE \$139.99

RATING ★ 4.7

POPULARITY 88/100

KNN SIMILARITY 24.6%

Logitech MX Master 3S

8K DPI sensor, MagSpeed scroll, silent clicks.

PRICE \$99.99

RATING ★ 4.8

POPULARITY 89/100

KNN SIMILARITY 21.3%

14

RESULTS AND FUTURE ENHANCEMENT

RESULTS

The developed product recommendation system using the K-Nearest Neighbors (KNN) algorithm was successfully implemented and tested using the selected dataset. The system is capable of generating relevant product recommendations based on similarity between product features such as price, rating, and category. When a user inputs a product, the model efficiently identifies the nearest neighbors and displays a list of similar products through the web interface.

The results show that the system provides accurate and meaningful recommendations for products that share similar characteristics. For example, when a user selects a product like a laptop, the system recommends other laptops with similar price ranges and ratings. The response time of the system is fast, making it suitable for real-time applications. The integration of the KNN model with the Flask-based web application ensures smooth interaction between the user interface and the backend processing.

Overall, the project demonstrates that the KNN algorithm is effective for building a simple and efficient recommendation system. The results confirm that similarity-based approaches can

FUTURE ENHANCEMENTS

The current product recommendation system can be further improved in several ways to enhance its performance, scalability, and user experience. One important enhancement is the implementation of a **hybrid recommendation system**, which combines KNN with other techniques such as collaborative filtering or deep learning models. This would improve the accuracy of recommendations by considering both product similarity and user behavior.

Another enhancement is the addition of **user personalization features**. By incorporating user login and maintaining user history, the system can provide more customized recommendations based on individual preferences, past purchases, and browsing patterns. This would make the system more dynamic and aligned with real-world e-commerce platforms.

The system can also be improved by handling **large-scale datasets** using advanced technologies such as big data frameworks. Optimizing the algorithm for faster computation and using dimensionality reduction techniques can help improve performance when dealing with a large number of products.

In addition, the **user interface can be enhanced** by improving the design of the web application, adding product images, filters, and sorting options. This would make the system more interactive and user-friendly.

Finally, the project can be extended by **deploying the application on cloud platforms** and integrating real-time data updates. This would allow the system to operate in real-world environments and continuously improve recommendations based on new data.

**Git Hub Link of the project
along with the report**

<https://github.com/Ajay0000777/Product-Recommendation-using-KNN.git>

REFERENCES

- Python Software Foundation, Python Documentation. Available at: <https://docs.python.org/>
- Scikit-learn Developers, Scikit-learn: Machine Learning in Python Documentation. Available at: <https://scikit-learn.org/>
- NumPy Developers, NumPy Documentation. Available at: <https://numpy.org/doc/>
- Pandas Development Team, Pandas Documentation. Available at: <https://pandas.pydata.org/docs/>
- Flask Development Team, Flask Web Framework Documentation. Available at: <https://flask.palletsprojects.com/>
- Chart.js Contributors, Chart.js Documentation. Available at: <https://www.chartjs.org/docs/>
- NSE India, National Stock Exchange Data Portal. Available at: <https://www.nseindia.com/>
- Trevor Hastie, Robert Tibshirani, and Jerome Friedman, *The Elements of Statistical Learning*. Springer, 2009.
- Kevin P. Murphy, *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012.
- Christopher M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- Ian Goodfellow, Yoshua Bengio, and Aaron Courville, *Deep Learning*. MIT Press, 2016.